

UNIT – 4
ASSESSMENT – 1
PYTHON CODE

NIHARIKA PREEJESH 192524506

TASK 1:

```
import numpy as np
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
# -----
```

```
# Create sample patient dataset
```

```
# -----
```

```
np.random.seed(42)
```

```
n = 200
```

```
data = pd.DataFrame({
    "Age": np.random.randint(20, 80, n),
    "BloodPressure": np.random.randint(90, 180, n),
    "Cholesterol": np.random.randint(120, 300, n),
```

```
"Glucose": np.random.randint(70, 220, n),  
"BMI": np.round(np.random.uniform(18, 45, n), 1),  
"FamilyHistory": np.random.randint(0, 2, n)  
})
```

```
# Create demonstration target
```

```
risk = (  
    0.03 * data["Age"] +  
    0.04 * data["BloodPressure"] +  
    0.02 * data["Cholesterol"] +  
    0.05 * data["Glucose"] +  
    0.08 * data["BMI"] +  
    0.8 * data["FamilyHistory"]  
)
```

```
probability = 1 / (1 + np.exp(-(risk - 13)))
```

```
data["Diabetes"] = (  
    np.random.random(n) < probability  
) .astype(int)
```

```
print("DATASET")
```

```
print(data.head())
```

```
# -----  
# Features and target  
# -----  
  
X = data.drop("Diabetes", axis=1)  
y = data["Diabetes"]  
  
print("\nTARGET VARIABLE: Diabetes")  
  
print("\nClass Distribution:")  
print(y.value_counts())  
  
# -----  
# 70:30 split  
# -----  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)
```

```
# -----  
# Standardization  
# -----  
  
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)  
  
print("\nDATA PREPARATION")  
print("-----")  
print("Total samples      :", len(data))  
print("Training samples   :", len(X_train))  
print("Testing samples    :", len(X_test))  
print("Training percentage : 70%")  
print("Testing percentage  : 30%")  
print("Scaling            : StandardScaler")  
print("Class balancing     : Class Weights")  
print("\nData preparation completed.")
```

TASK 2:

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    precision_score,
```

```
    recall_score,
```

```
    f1_score,
```

```
    confusion_matrix
```

```
)
```

```
# -----
```

```
# Create dataset
```

```
# -----
```

```
np.random.seed(42)
```

```
n = 300
```

```
data = pd.DataFrame({
```

```
    "Age": np.random.randint(20, 80, n),
```

```

    "BloodPressure": np.random.randint(90, 180, n),
    "Cholesterol": np.random.randint(120, 300, n),
    "Glucose": np.random.randint(70, 220, n),
    "BMI": np.round(np.random.uniform(18, 45, n), 1),
    "FamilyHistory": np.random.randint(0, 2, n)
})

```

```

risk = (
    0.03 * data["Age"] +
    0.04 * data["BloodPressure"] +
    0.02 * data["Cholesterol"] +
    0.05 * data["Glucose"] +
    0.08 * data["BMI"] +
    0.8 * data["FamilyHistory"]
)

```

```

probability = 1 / (1 + np.exp(-(risk - 13)))

```

```

data["Diabetes"] = (
    np.random.random(n) < probability
).astype(int)

```

```

# -----

```

```

# Split data

```

```
# -----
```

```
X = data.drop("Diabetes", axis=1)
```

```
y = data["Diabetes"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)
```

```
# -----
```

```
# Decision Tree
```

```
# -----
```

```
tree = DecisionTreeClassifier(  
    criterion="gini",  
    max_depth=4,  
    class_weight="balanced",  
    random_state=42  
)
```

```
tree.fit(X_train, y_train)
```

```
prediction = tree.predict(X_test)

# -----
# Evaluation
# -----

accuracy = accuracy_score(y_test, prediction)
precision = precision_score(
    y_test, prediction, zero_division=0
)
recall = recall_score(
    y_test, prediction, zero_division=0
)
f1 = f1_score(
    y_test, prediction, zero_division=0
)

print("DECISION TREE PERFORMANCE")
print("-----")
print("Accuracy :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 Score :", round(f1, 4))
```

```

# -----
# Confusion Matrix
# -----

cm = confusion_matrix(y_test, prediction)

print("\nCONFUSION MATRIX")
print(cm)

tn, fp, fn, tp = cm.ravel()

print("\nTrue Negatives :", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives :", tp)

# -----
# First three levels
# -----

print("\nFIRST THREE LEVELS")
print("-----")

```

```
print(  
    export_text(  
        tree,  
        feature_names=list(X.columns),  
        max_depth=3  
    )  
)
```

```
# -----  
# Feature Importance  
# -----
```

```
importance = pd.Series(  
    tree.feature_importances_,  
    index=X.columns  
)  
.sort_values(ascending=False)
```

```
print("\nFEATURE IMPORTANCE")  
print(importance)
```

```
# -----  
# Post Pruning  
# -----
```

```
unpruned = DecisionTreeClassifier(  
    class_weight="balanced",  
    random_state=42  
)  
  
unpruned.fit(X_train, y_train)  
  
unpruned_pred = unpruned.predict(X_test)  
  
unpruned_accuracy = accuracy_score(  
    y_test, unpruned_pred  
)  
  
path = unpruned.cost_complexity_pruning_path(  
    X_train,  
    y_train  
)  
  
alphas = path.ccp_alphas  
  
if len(alphas) > 2:  
    alpha = alphas[len(alphas) // 2]  
else:  
    alpha = 0
```

```
pruned = DecisionTreeClassifier(  
    class_weight="balanced",  
    ccp_alpha=alpha,  
    random_state=42  
)  
  
pruned.fit(X_train, y_train)  
  
pruned_pred = pruned.predict(X_test)  
  
pruned_accuracy = accuracy_score(  
    y_test, pruned_pred  
)  
  
print("\nPRUNING COMPARISON")  
print("-----")  
print(  
    "Pre/Post terminology:"  
)  
print(  
    "Pre-pruned Tree Accuracy :",  
    round(accuracy, 4)  
)
```

```
print(  
    "Post-pruned Tree Accuracy:",  
    round(pruned_accuracy, 4)  
)
```

TASK 3:

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    precision_score,
```

```
    recall_score,
```

```
    f1_score,
```

```
    roc_auc_score
```

```
)
```

```
# -----
```

```
# Dataset
```

```
# -----
```

```
np.random.seed(42)
```

```
n = 300
```

```
data = pd.DataFrame({  
    "Age": np.random.randint(20, 80, n),  
    "BloodPressure": np.random.randint(90, 180, n),  
    "Cholesterol": np.random.randint(120, 300, n),  
    "Glucose": np.random.randint(70, 220, n),  
    "BMI": np.round(np.random.uniform(18, 45, n), 1),  
    "FamilyHistory": np.random.randint(0, 2, n)  
})
```

```
risk = (  
    0.03 * data["Age"] +  
    0.04 * data["BloodPressure"] +  
    0.02 * data["Cholesterol"] +  
    0.05 * data["Glucose"] +  
    0.08 * data["BMI"] +  
    0.8 * data["FamilyHistory"]  
)
```

```
probability = 1 / (1 + np.exp(-(risk - 13)))
```

```
data["Diabetes"] = (  
    np.random.random(n) < probability  
) .astype(int)
```

```
# -----
```

```
# Split
```

```
# -----
```

```
X = data.drop("Diabetes", axis=1)
```

```
y = data["Diabetes"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)
```

```
# -----
```

```
# Standardization
```

```
# -----
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# -----
```

```
# Logistic Regression
```

```
# -----
```

```
model = LogisticRegression(
```

```
    class_weight="balanced",
```

```
    max_iter=1000,
```

```
    random_state=42
```

```
)
```

```
model.fit(X_train_scaled, y_train)
```

```
prediction = model.predict(X_test_scaled)
```

```
probability_prediction = model.predict_proba(
```

```
    X_test_scaled
```

```
)[:, 1]
```

```
# -----
```

```
# Evaluation
```

```
# -----
```

```
accuracy = accuracy_score(  
    y_test, prediction  
)
```

```
precision = precision_score(  
    y_test, prediction,  
    zero_division=0  
)
```

```
recall = recall_score(  
    y_test, prediction,  
    zero_division=0  
)
```

```
f1 = f1_score(  
    y_test, prediction,  
    zero_division=0  
)
```

```
auc = roc_auc_score(  
    y_test,  
    probability_prediction
```

)

```
print("LOGISTIC REGRESSION")
print("-----")
print("Accuracy :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 Score :", round(f1, 4))
print("AUC-ROC  :", round(auc, 4))
```

```
# -----
# Feature Importance
# -----
```

```
importance = pd.Series(
    abs(model.coef_[0]),
    index=X.columns
).sort_values(ascending=False)
```

```
print("\nFEATURE IMPORTANCE")
print("-----")
print(importance)
```

```
print("\nTOP THREE PREDICTORS")
```

```
print("-----")
```

```
for feature in importance.head(3).index:
```

```
    print(feature)
```

```
# -----
```

```
# Compare with Decision Tree
```

```
# -----
```

```
print("\nMODEL INTERPRETATION")
```

```
print("-----")
```

```
print(
```

```
    "The Logistic Regression model uses coefficient magnitude"
```

```
)
```

```
print(
```

```
    "to estimate the relative importance of features."
```

```
)
```

TASK 4:

```
import numpy as np
```

```
import pandas as pd
```

```
# -----
```

```
# STATES AND ACTIONS
```

```
# -----
```

```
states = [
```

```
    "Low_Risk",
```

```
    "Moderate_Risk",
```

```
    "High_Risk",
```

```
    "Critical_Risk"
```

```
]
```

```
actions = [
```

```
    "Diet",
```

```
    "Exercise",
```

```
    "Medication",
```

```
    "Monitor"
```

```
]
```

```
# -----
```

```
# Q TABLE
```

```
# -----
```

```
Q = pd.DataFrame(  
    0.0,  
    index=states,  
    columns=actions  
)
```

```
# Learning parameters
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 0.2
```

```
# -----
```

```
# TRANSITION FUNCTION
```

```
# -----
```

```
def next_state(state, action):
```

```
    transition = {
```

```
        "Low_Risk": {
```

```
            "Diet": "Low_Risk",
```

```
"Exercise": "Low_Risk",  
"Medication": "Moderate_Risk",  
"Monitor": "Moderate_Risk"  
},
```

```
"Moderate_Risk": {  
  "Diet": "Low_Risk",  
  "Exercise": "Low_Risk",  
  "Medication": "Moderate_Risk",  
  "Monitor": "High_Risk"  
},
```

```
"High_Risk": {  
  "Diet": "Moderate_Risk",  
  "Exercise": "Moderate_Risk",  
  "Medication": "Low_Risk",  
  "Monitor": "Critical_Risk"  
},
```

```
"Critical_Risk": {  
  "Diet": "High_Risk",  
  "Exercise": "High_Risk",  
  "Medication": "Moderate_Risk",  
  "Monitor": "Critical_Risk"
```

```
    }  
}
```

```
return transition[state][action]
```

```
# -----
```

```
# REWARD FUNCTION
```

```
# -----
```

```
def reward_function(state, action, new_state):
```

```
    values = {  
        "Low_Risk": 3,  
        "Moderate_Risk": 2,  
        "High_Risk": 1,  
        "Critical_Risk": 0  
    }
```

```
    reward = values[new_state] - values[state]
```

```
    if state == "Critical_Risk" and action == "Medication":
```

```
        reward += 3
```

```
return reward
```

```
# -----
```

```
# Q LEARNING
```

```
# -----
```

```
episodes = 10
```

```
np.random.seed(42)
```

```
for episode in range(1, episodes + 1):
```

```
    state = "High_Risk"
```

```
    print("\nEpisode", episode)
```

```
    for step in range(5):
```

```
        # Exploration / exploitation
```

```
        if np.random.random() < epsilon:
```

```
            action = np.random.choice(actions)
```

```
        else:
```

```
action = Q.loc[state].idxmax()
```

```
new_state = next_state(  
    state,  
    action  
)
```

```
reward = reward_function(  
    state,  
    action,  
    new_state  
)
```

```
old_q = Q.loc[state, action]
```

```
best_future_q = Q.loc[new_state].max()
```

```
new_q = old_q + alpha * (  
    reward +  
    gamma * best_future_q -  
    old_q  
)
```

```
Q.loc[state, action] = new_q
```

```
# Show iterations  
if episode <= 2 and step < 3:
```

```
    print(  
        "State:", state,  
        "| Action:", action,  
        "| Reward:", reward,  
        "| Next:", new_state,  
        "| Q:", round(new_q, 3)  
    )
```

```
    state = new_state
```

```
# -----
```

```
# FINAL Q TABLE
```

```
# -----
```

```
print("\nFINAL Q TABLE")
```

```
print("-----")
```

```
print(Q.round(3))
```

```

# -----
# FINAL POLICY
# -----

print("\nFINAL LEARNED POLICY")
print("-----")

for state in states:

    best_action = Q.loc[
        state
    ].idxmax()

    print(
        state,
        "->",
        best_action
    )

```